



<UI Components>

[UNIFEI | Universidade Federal de Itajubá](#)

Instituto de Ciências Tecnológicas

ECO – <Dev. Mobile>

eduardo.felipe@unifei.edu.br

"Apaga a imaginação. Imobiliza a movimentação dos cordões. Circunscreve-te ao tempo presente. Aprende a conhecer o que acontece a ti ou aos outros. Discerne e decompõe no que serve de fundamento às coisas o causal e o material. Pensa na hora derradeira. Que o erro cometido por alguém permaneça onde foi cometido." - Meditações (Marco Aurélio)

Components

<https://reactnative.dev/docs/components-and-apis>

ActivityIndicator

```
import { View, Text, ActivityIndicator, ToastAndroid } from 'react-native'
import React, { useEffect, useState } from 'react'

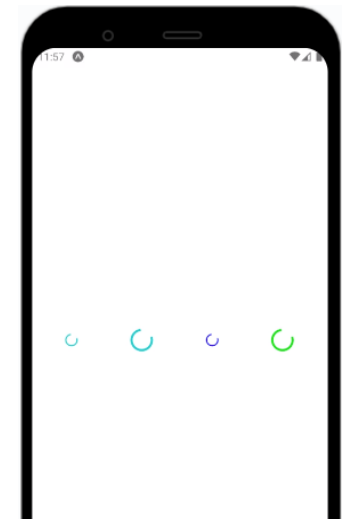
export default function Tela8() {
  const [wait, setWait] = useState(false);
  useEffect(() => {
    setTimeout(() => {
      ToastAndroid.show("Fazendo algo demorado...", ToastAndroid.SHORT);
      setWait(true);
    }, 1500);
    setTimeout(() => {
      ToastAndroid.show("Terminando algo demorado...", ToastAndroid.SHORT);
      setWait(false);
    }, 5000);

  }, []);
}
```



ActivityIndicator

```
return (  
  <View style={{ flex: 1, justifyContent: "center" }}>  
    <ActivityIndicator  
      size={"large"}  
      animating={wait} />  
    {!wait ? <Text>Alguma interface</Text> : false}  
  </View>  
)  
}
```



FlatList com *List/Item*

<https://randomuser.me/>

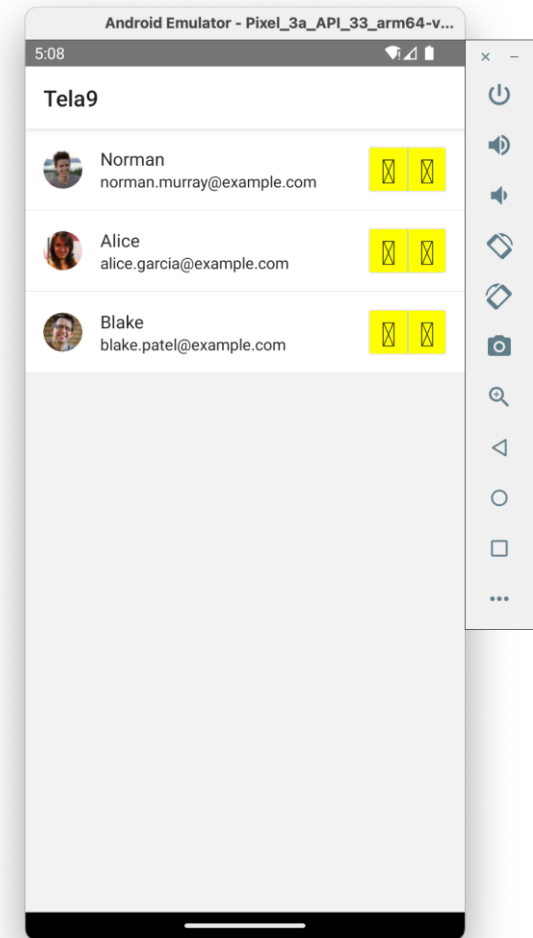
<https://reactnativeelements.com/docs/installation>

FlatList (com React Native Elements)

- Componente importante para listar itens em interfaces para dispositivos móveis.
 - Fully cross-platform.
 - Optional horizontal mode.
 - Configurable viewability callbacks.
 - Header support.
 - Footer support.
 - Separator support.
 - Pull to Refresh.
 - **Scroll loading.**
 - ScrollToIndex support.
 - Multiple column support.

<https://reactnativeelements.com/docs/components/listitem>

<https://reactnative.dev/docs/flatlist>



FlatList com ListItem

```
import React from 'react';  
import {View, FlatList, Alert} from 'react-native';  
import { SafeAreaView } from 'react-native-safe-area-context';  
import { Avatar, ListItem, Icon } from '@rneui/base';  
  
import { USERS } from './DataUsers'
```

Obs: <https://randomuser.me/api/?results=3>



FlatList com ListItem

```
export default function explore() {

  const [USERS, setUSERS] = useState([]);

  useEffect(() => {
    fetch("https://randomuser.me/api/?results=5")
      .then((respFetch) => respFetch.json())
      .then((respJson) => setUSERS(respJson))
      .catch((erro) => Alert.alert("Ocorreu o erro: ", erro))
  }, [])

  return (
    <SafeAreaView style={{flex: 1}}>
      <FlatList
        data={USERS.results}
        keyExtractor={item => item.login.uuid}
        renderItem={({ item }) => getItem(item)}
      />
    </SafeAreaView>
  );
}
```




```
function getItem(dado) {  
  return (  
    <ListItem  
      bottomDivider  
      onPress={() => Alert.alert("Teste " + dado.name.first)}>  
    <Avatar  
      size={35}  
      rounded  
      source={{ uri: dado.picture.thumbnail }} />  
    <ListItem.Content style={{ flex: 1, flexDirection: 'row', justifyContent: 'space-between' }}>  
      <View>  
        <ListItem.Title>{dado.name.first}</ListItem.Title>  
        <ListItem.Subtitle>{dado.email}</ListItem.Subtitle>  
      </View>  
      <View style={{width: 70}}>  
        <ListItem.ButtonGroup  
          buttonStyle={{backgroundColor: 'yellow'}}  
          selectedButtonStyle={{ backgroundColor: '#e2e2e2' }}  
          buttons={[  
            <Icon size={30} name="edit" onPress={() => Alert.alert("icone 1" + dado.login.uuid)} />,  
            <Icon size={30} name="delete" onPress={() => Alert.alert("icone 2 " + dado.name.first + " " + dado.id.value)} />,  
          ]}  
        />  
      </View>  
    </ListItem.Content>  
  </ListItem>  
)  
}
```



FlatList

Sem memória de seleção

FlatList (“memória” de seleção)

```
import { View, Text, FlatList, StyleSheet, Button } from 'react-native'
```

```
import React, { useState } from 'react'
```

```
export default function Tela10(props) {  
  const [itemSelect, setItemSelect] = useState(-1);
```

```
  const OPTIONS = [
```

```
    {  
      id: '7',  
      title: "opção 7"
```

```
    },
```

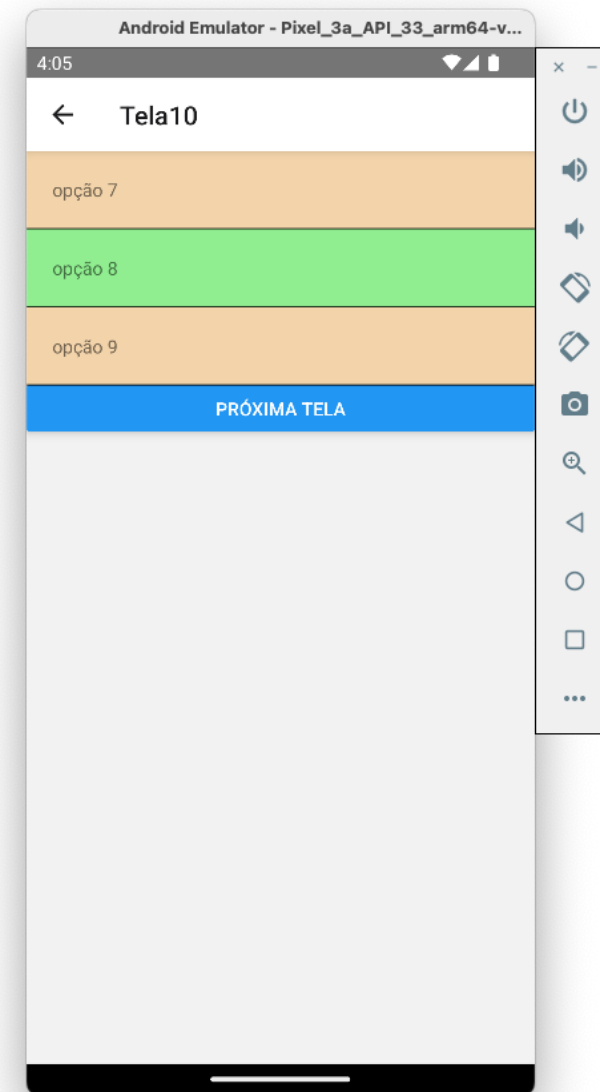
```
    {  
      id: '8',  
      title: "opção 8"
```

```
    },
```

```
    {  
      id: '9',  
      title: "opção 9"
```

```
    },
```

```
  ]
```



FlatList (“memória” de seleção)

```
return (  
  <View>  
    <FlatList  
      data={OPTIONS}  
      keyExtractor={item => item.id}  
      extraData={itemSelect}  
      renderItem={({ item }) => (  
        <Text  
          style={item.id === itemSelect ? styles.selected : styles.item}  
          onPress={() => setItemSelect(item.id)}>{item.title}</Text>  
        )}  
      >  
    </FlatList>  
    <Button onPress={() => props.navigation.navigate('Tela8')} title={"Próxima Tela"} />  
  </View>  
)  
}
```



FlatList (“memória” de seleção)

```
const styles = StyleSheet.create({
  item: {
    backgroundColor: '#f2d3aa',
    padding: 20,
    borderStyle: 'solid',
    borderBottomWidth: 1,
  },
  selected: {
    backgroundColor: 'lightgreen',
    padding: 20,
    borderStyle: 'solid',
    borderBottomWidth: 1,
  }
})
```



FlatList (“memória” de seleção)

- Há um problema com a lista anterior:
 - Vá para próxima tela e volte;
 - Vá para tela anterior e avance;
- O que acontece nestes cenários com o item selecionado na FlatList?



Tipografia

Fonts padronizadas - CLI

display4	Light 112
display3	Regular 56
display2	Regular 45
display1	Regular 34
headline	Regular 24
title	Medium 20
subheading	Regular 16
body2	Medium 14
body1	Regular 14
caption	Regular 12
button	MEDIUM 14



Fonts - Expo

- <https://docs.expo.dev/develop/user-interface/fonts/>



Formato monetário

Formatação monetária - CLI

<https://github.com/CaioQuirinoMedeiros/react-native-currency-input#readme>

CurrencyInput Examples

123456.78 0 null -9257.863942

1	2	3	-
4	5	6	⌋
7	8	9	✕
,	0	.	✓



Formatação monetária - Expo

- <https://www.npmjs.com/package/react-native-format-currency>

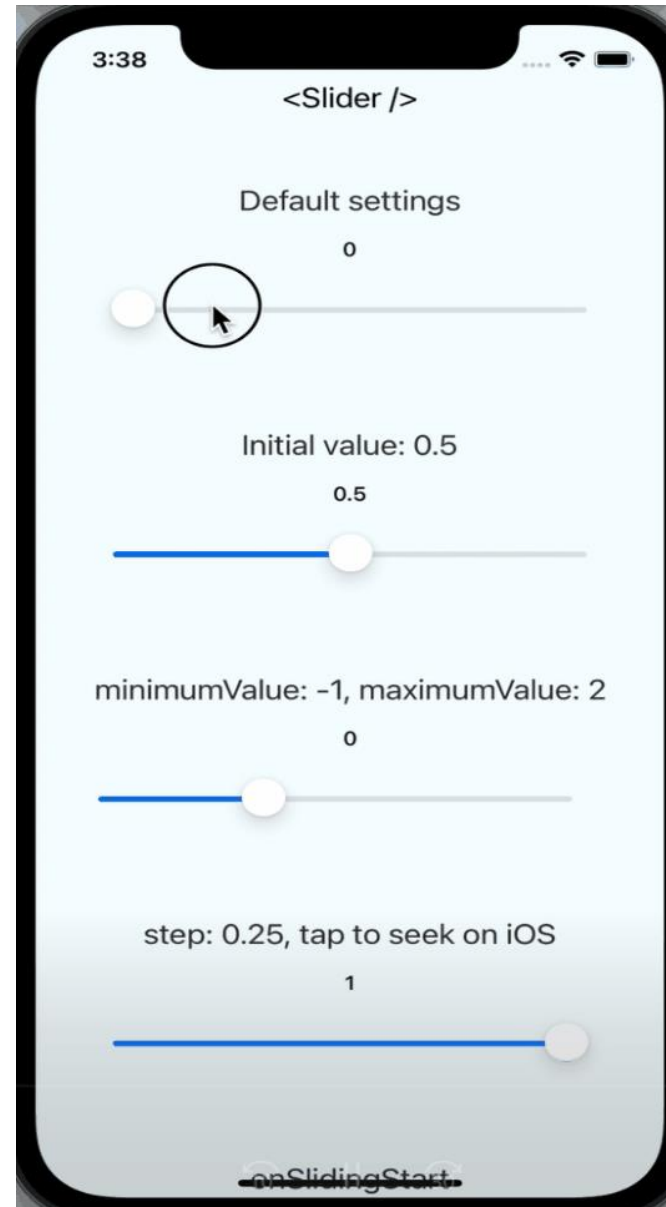


Slider

Slider - CLI

<https://www.npmjs.com/package/@react-native-community/slider>

<https://github.com/callstack/react-native-slider#readme>



Slider - Expo

- <https://docs.expo.dev/versions/latest/sdk/slider/>



Linking

<https://reactnative.dev/docs/linking>


```
import { View, Text, Button, Linking } from 'react-native'
import React from 'react'
```

```
export default function linking() {
```

```
  async function sendWhats(msg) {
    await Linking.openURL(`https://wa.me/?text=${msg}`);
  }
```

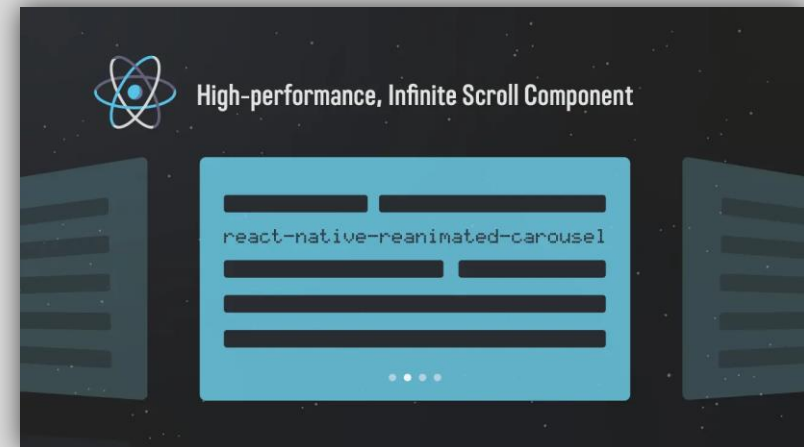
```
  return (
    <View>
      <Text>linking</Text>
      <Button title='Envia texto' onPress={() => sendWhats("Texto exemplo, mensagem
qualquer.")} />
    </View>
  )
}
```



Exemplo

Bibliotecas Externas

React Native Reanimated Carousel



Instalação

- npm install react-native-reanimated-carousel
- npm install --save react-native-gesture-handler
- npm install react-native-reanimated

- (cli) npm start -- --reset-cache
- (expo) npx expo start -c

- (mac) np**x** pod-install

Add `react-native-reanimated/plugin` plugin to your `babel.config.js`.

```
module.exports = {
  presets: [
    ... // don't add it here :)
  ],
  plugins: [
    ...
    'react-native-reanimated/plugin',
  ],
};
```

<https://rn-carousel.dev>

<https://docs.swmansion.com/react-native-gesture-handler/docs/fundamentals/installation>

<https://docs.swmansion.com/react-native-reanimated/docs/fundamentals/getting-started/#installation>



React Native Reanimated Carousel

```
import { View, Dimensions, Image } from 'react-native'
import React from 'react'
import Carousel from 'react-native-reanimated-carousel'

export default function Tela7() {
  const images = [
    { id: '1', image: require('../assets/images/react-logo.png') },
    { id: '2', image: require('../assets/images/partial-react-logo.png') },
    { id: '3', image: require('../assets/images/favicon.png') } ]

  const width = Dimensions.get('window').width;
  const height = Dimensions.get('screen').height;
```



```
return (  
  <View style={{ flex: 1 }}>  
    <Carousel  
      loop={false}  
      width={width}  
      height={height / 2}  
      autoPlay={true}  
      data={images}  
      scrollAnimationDuration={1000}  
      onSnapToItem={(index) => console.log('current index:', index)}  
      renderItem={({ item, index }) => (  
        <View  
          style={{ flex: 1, borderWidth: 0 }}>  
            <Image  
              key={index}  
              style={{ width: '100%', height: '100%' }}  
              resizeMode='contain'  
              source={item.image} />  
            </View>  
          )}  
        />  
      </View>  
    )  
  }  
)
```



Outras bibliotecas

- <https://www.npmjs.com/package/react-native-webview>
- <https://www.npmjs.com/package/react-native-pdf>



Frameworks

Conjunto de componentes

- React Native Elements
 - <https://reactnativeelements.com>
- Gluestack
 - <https://ui.gluestack.io/>
- React Native Inkpad
 - <https://rn-inkpad.web.app>



Design

<https://github.com/Aashu-Dubey/React-Native-UI-Templates>

<https://mobbin.com>



Referências

Referências

- <https://ui.gluestack.io/>
- <https://github.com/dohooo/react-native-reanimated-carousel>
- <https://www.crowdbotics.com/blog/how-to-create-a-custom-gallery-view-in-react-native>
- <https://reactnative.directory/>
- <https://www.npmjs.com/package/react-native-device-info#react-native-device-info>



Referências

- <https://reactnativeelements.com>
- <https://youtu.be/MnHZ0jtGsc8?si=K9eglze83ugfYRLn>



Seja esse cara...





•

eduardo.felipe@unifei.edu.br